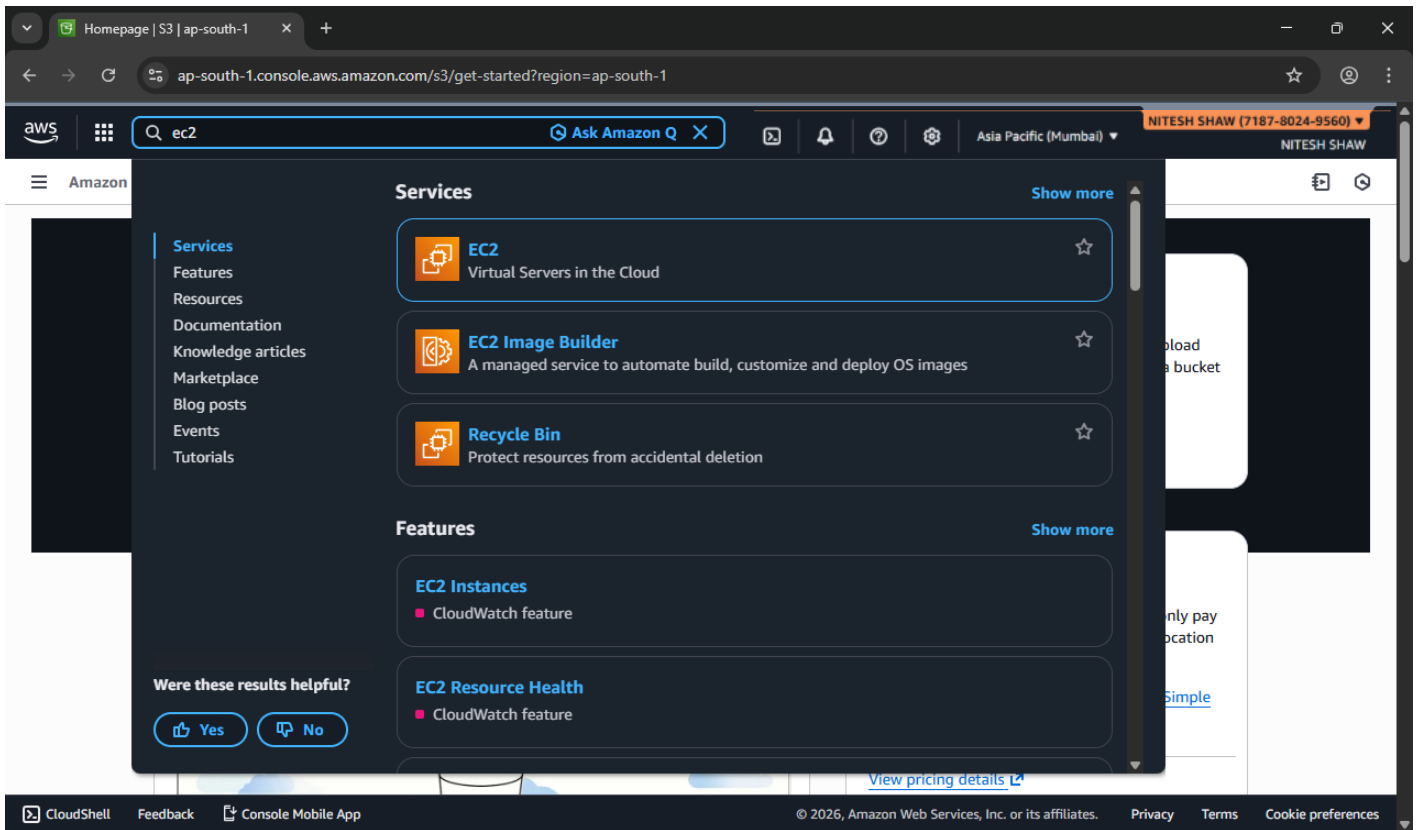


Assignment No: 12

Problem Statement: Deploy and run the project in AWS without using the port.

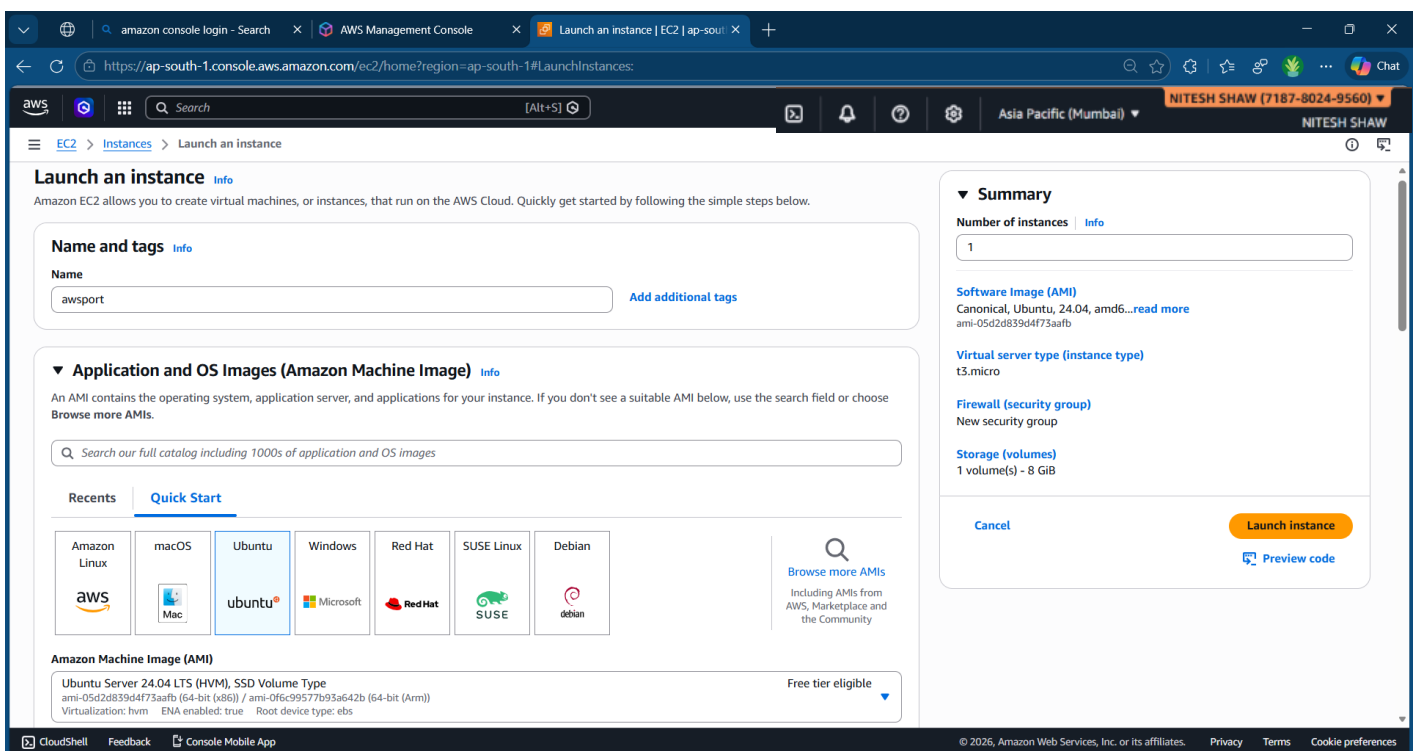
Step 1: Open EC2 Service

Log in to the AWS Management Console. Search for **EC2** in the top search bar and open the **Amazon EC2** service. This service allows you to create and manage virtual servers.



Step 2: Launch a New EC2 Instance

Click **Launch Instance**. Provide a name for the instance (Ec2-Instance). Choose an **Amazon Machine Image (AMI)** such as **Amazon Ubuntu**. Select an **Instance Type** (t3.micro, free-tier eligible).



Step 3: Choose a **key pair** name from the existing. Choose **Select existing security group** and choose one of the existing groups in this case **sicky9304**.

Key pair (login) Info

You can use a key pair to securely connect to your instance. Ensure that you have access to the selected key pair before you launch the instance.

Key pair name - **required**

key4 [Create new key pair](#)

Network settings Info [Edit](#)

Network Info

vpc-071938336c919d28d

Subnet Info

No preference (Default subnet in any availability zone)

Auto-assign public IP Info

Enable

Firewall (security groups) Info

A security group is a set of firewall rules that control the traffic for your instance. Add rules to allow specific traffic to reach your instance.

☐ Create security group ☒ **Select existing security group**

Common security groups Info

Select security groups

sicky9304 sg-060730298a9c60064 [X](#)

VPC: vpc-071938336c919d28d

Security groups that you add or remove here will be added to or removed from all your network interfaces.

[Compare security group rules](#)

Summary

Number of instances [Info](#)

1

Software Image (AMI)

Canonical, Ubuntu, 24.04, amd64...[read more](#)

ami-05d2d839d4f73aafb

Virtual server type (instance type)

t3.micro

Firewall (security group)

sicky9304

Storage (volumes)

1 volume(s) - 8 GiB

[Cancel](#) [Launch instance](#) [Preview code](#)

Step 4: In the user data field enter the following script and click on **Launch Instance** button.

Metadata response hop limit Info

2

Allow tags in metadata Info

Select

User data - optional Info

Upload a file with your user data or enter it in the field.

[Choose file](#)

```
#!/bin/bash
apt-get update
apt-get install -y nginx
systemctl start nginx
systemctl enable nginx
apt-get install -y git
sudo curl -sL https://deb.nodesource.com/setup_18.x | sudo -E bash -
apt-get install -y nodejs
git clone https://github.com/Nitesh12/demo-app
cd aws-repo
npm install
node index.js
```

☐ User data has already been base64 encoded

Summary

Number of instances [Info](#)

1

Software Image (AMI)

Canonical, Ubuntu, 24.04, amd64...[read more](#)

ami-05d2d839d4f73aafb

Virtual server type (instance type)

t3.micro

Firewall (security group)

sicky9304

Storage (volumes)

1 volume(s) - 8 GiB

[Cancel](#) [Launch instance](#) [Preview code](#)

Step 5: A new Instance will be successfully created.

Success

Successfully initiated launch of instance (i-0aaefa6dd7f5534a0)

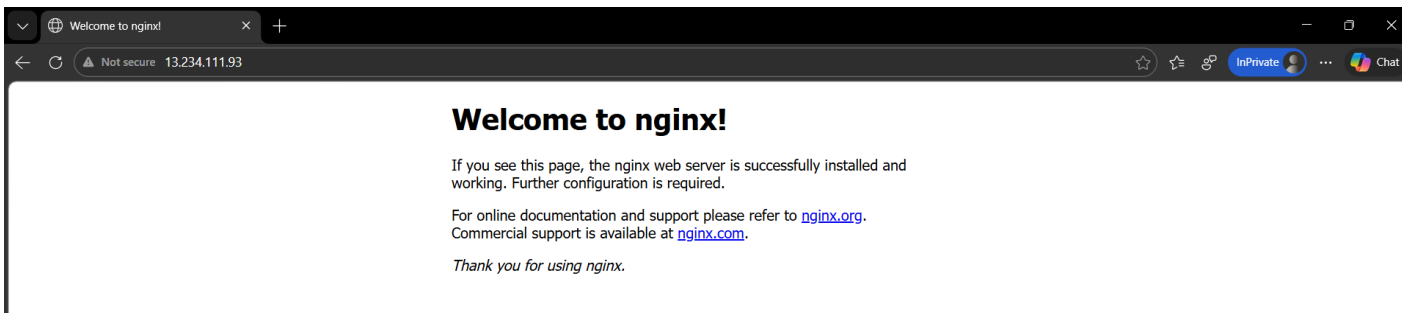
Launch log

Next Steps

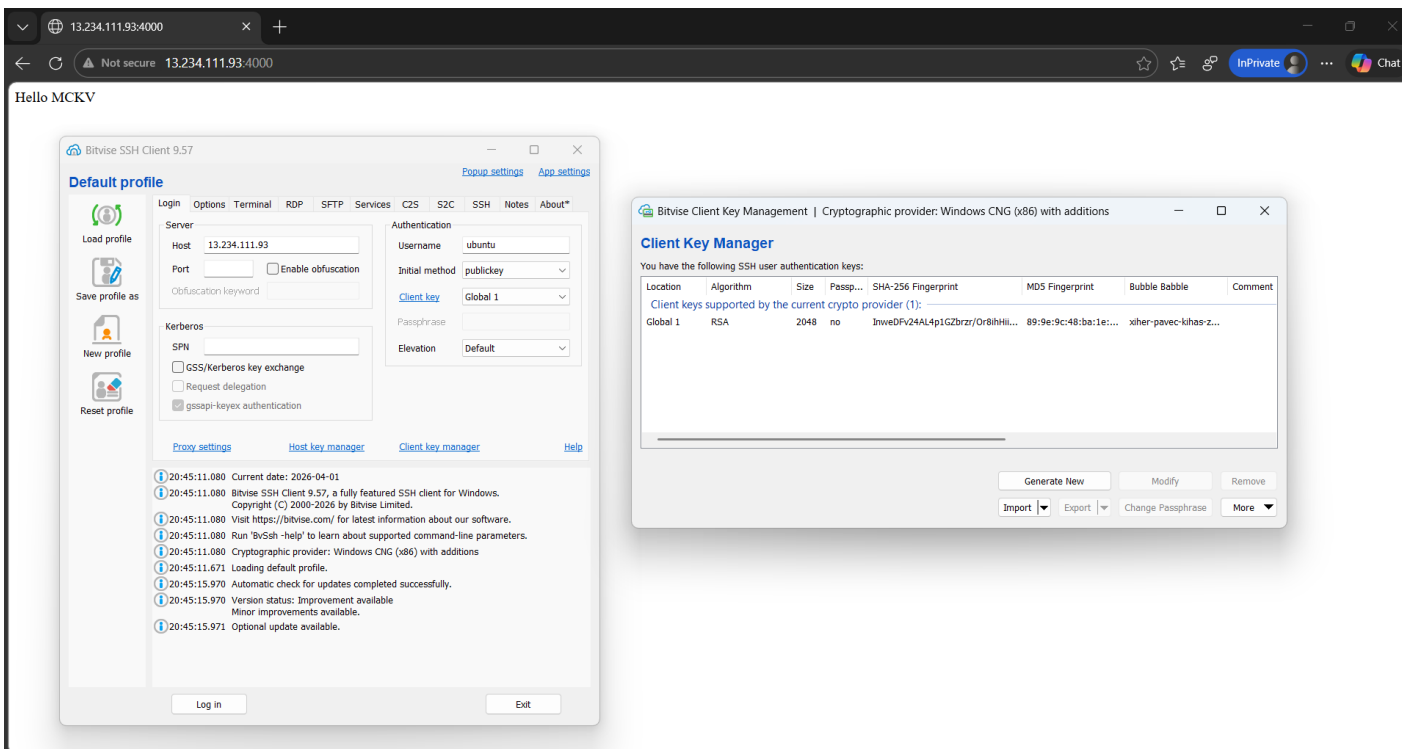
What would you like to do next with this instance, for example "create alarm" or "create backup"

< 1 2 3 4 5 6 >

Step 6: We can see the instance is running by going to the IPv4 link and with :4000 port.



Step 7: Open bitwise, and enter the Host, Username as **ubuntu**, initial method as **public key** and click on the **Log in** button.



Step 8: In the terminal, enter the commands **cd /** to go to the root, **ls** to view the files, then **cd etc**, **cd nginx/**, **cd sites-available/** to go to the required directory.

ubuntu@13.234.111.93:22 - Bitwise xterm - ubuntu@ip-172-31-43-232: /etc/nginx/sites-available

```
ubuntu@ip-172-31-43-232:~$ cd /
ubuntu@ip-172-31-43-232:/$ ls
bin          dev          lib.usr-is-merged  mnt          run          srv          var
bin.usr-is-merged  etc          lib64             opt          sbin         sys
boot         home         lost+found        proc         sbin.usr-is-merged  tmp
demo-app     lib          media             root         snap         usr

ubuntu@ip-172-31-43-232:/$ cd etc
ubuntu@ip-172-31-43-232:/etc$ cd nginx/
ubuntu@ip-172-31-43-232:/etc/nginx$ cd sites-available/
ubuntu@ip-172-31-43-232:/etc/nginx/sites-available$
```

Step 9: Enter the command `sudo nano default` to edit the default file.

```
ubuntu@ip-172-31-43-232:/etc/nginx/sites-available$ ls
default
ubuntu@ip-172-31-43-232:/etc/nginx/sites-available$ sudo nano default
```

Step 10: In the file edit the location `/ {}` and write the following.

```
location / {
    # First attempt to serve request as file, then
    # as directory, then fall back to displaying a 404.
    #try_files $uri $uri/ =404;
    proxy_pass http://localhost:4000;
    proxy_http_version 1.1;
    proxy_set_header Upgrade $http_upgrade;
    proxy_set_header Connection 'upgrade';
    proxy_set_header Host $host;
    proxy_cache_bypass $http_upgrade;
}
```

Step 11: Enter the command `sudo chmod +x default` to give the file the necessary permissions.

```
ubuntu@ip-172-31-43-232:/etc/nginx/sites-available$ sudo chmod +x default
```

Step 12: Enter the command `cd ..` to return to the `etc/` folder and then enter the command `sudo systemctl restart nginx` for saving make changes and restart the nginx server.

```
ubuntu@ip-172-31-43-232:/etc/nginx/sites-available$ cd ..
ubuntu@ip-172-31-43-232:/etc/nginx$ cd ..
ubuntu@ip-172-31-43-232:/etc$ sudo systemctl restart nginx
```

Step 13: enter the command `sudo systemctl enable nginx` makes Nginx start automatically on system boot, so whenever your EC2 instance restarts, Nginx will start itself, no need to manually run start again.

```
ubuntu@ip-172-31-43-232:/etc$ sudo systemctl enable nginx
Synchronizing state of nginx.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable nginx
ubuntu@ip-172-31-43-232:/etc$
```

Step 14: The project will be deployed and run without a port.

